

Módulo 6 – Capítulo 02 – Sección 01

Qué son los embeddings y cómo capturan significado semántico

Cuando un sistema RAG recibe la query "¿cuáles son los efectos secundarios del ibuprofeno en pacientes con insuficiencia renal?", necesita encontrar entre millones de fragmentos de texto los que contienen la respuesta. Esa búsqueda no puede basarse en palabras clave exactas: el documento médico relevante puede hablar de "nefrotoxicidad por AINEs" sin mencionar "ibuprofeno" ni "insuficiencia renal" literalmente. Los embeddings son la tecnología que hace posible esta búsqueda semántica: representaciones numéricas del texto en un espacio geométrico donde la proximidad entre vectores refleja la similitud de significado entre textos, independientemente de las palabras específicas usadas.

Un embedding es, en términos técnicos, una proyección de un objeto —texto, imagen, audio— a un vector de números reales en un espacio de alta dimensionalidad (típicamente 256 a 3072 dimensiones). Lo que hace a esta proyección útil es que no es arbitraria: está aprendida por una red neuronal entrenada para que textos con significados similares produzcan vectores cercanos en ese espacio, y textos con significados distintos produzcan vectores distantes. La intuición matemática es que el espacio de embeddings tiene estructura: la dirección de "Rey - Hombre + Mujer $\approx$ Reina" es el ejemplo clásico de que las dimensiones del espacio vectorial codifican relaciones semánticas, no de forma explícita sino como patrones emergentes del entrenamiento.

Los modelos de embedding para texto —en particular los de la familia Sentence Transformers, formalizada por Reimers y Gurevych en el paper "Sentence-BERT" de 2019— son redes neuronales basadas en arquitecturas Transformer (BERT, RoBERTa) que reciben como entrada una secuencia de tokens y producen como salida un único vector que representa semánticamente toda la secuencia. El mecanismo clave es el pooling: los transformers producen una representación por token, y el pooling (típicamente mean pooling sobre todos los tokens de la secuencia) reduce esa secuencia de vectores a un único vector de documento. El entrenamiento utiliza una pérdida contrastiva: el modelo recibe pares de frases

semánticamente equivalentes (positivos) y pares de frases no relacionadas (negativos), y ajusta sus pesos para acercar los positivos y alejar los negativos en el espacio vectorial.

En el contexto específico de RAG, los embeddings sirven dos propósitos complementarios en la misma arquitectura: indexar los chunks del corpus durante la ingesta (generando el vector de cada fragmento de texto que se almacenará en la base vectorial) y procesar las queries del usuario en tiempo de inferencia (generando el vector de búsqueda que se comparará contra los vectores del índice). El resultado de esa comparación —la similitud coseno entre el vector de la query y el vector de cada chunk— es el score de relevancia que determina qué fragmentos se recuperan. Por esta razón, es imperativo usar el mismo modelo de embedding en la indexación y en el serving: un vector generado por text-embedding-3-small no es comparable geométricamente con un vector generado por voyage-3, incluso si ambos tienen 1024 dimensiones.

Conceptos técnicos fundamentales

- **Espacio latente:** espacio vectorial de D dimensiones (típicamente 256–3072) donde cada punto representa un texto; la geometría del espacio codifica relaciones semánticas aprendidas del corpus de preentrenamiento; la distancia coseno en este espacio mide similitud semántica.
- **Encoder transformer:** arquitectura basada en BERT/RoBERTa que procesa el texto de entrada token por token con atención multi-cabeza, generando representaciones contextualizadas donde el vector de cada token depende de todos los otros tokens de la secuencia.
- **Pooling de secuencia:** operación que reduce la secuencia de vectores de tokens a un único vector de documento; mean pooling (promedio de todos los vectores de tokens) es la estrategia más robusta en Sentence-BERT y sus variantes; CLS token pooling (usar solo el vector del token de clasificación) es más rápido pero generalmente inferior.
- **Entrenamiento contrastivo:** objetivo de pérdida (InfoNCE loss, Multiple Negatives Ranking loss) que maximiza la similitud coseno de pares positivos y minimiza la de pares negativos en el batch; requiere pares de texto semánticamente equivalentes para el conjunto de entrenamiento.
- **Similitud coseno:** métrica de distancia normalizada en rango $[-1, 1]$ que mide el coseno del ángulo entre dos vectores; es invariante a la magnitud del vector, lo que la hace adecuada para comparar textos de longitudes muy diferentes sin sesgo por longitud.
- **Sentence Transformers:** librería Python (pip install sentence-transformers) que implementa los modelos de la familia SBERT con una API simple; permite generar

embeddings con `model.encode(texts)` para listas de textos en batch, y soporta inferencia en CPU y GPU.

Nota del Arquitecto: *Un error conceptual frecuente es asumir que "más dimensiones = mejor calidad". La relación no es lineal: modelos de 1536 dimensiones pueden ser superiores a modelos de 3072 en dominios específicos si fueron entrenados con mejores datos de ese dominio. Y modelos de 256 dimensiones con Matryoshka Representation Learning (MRL) pueden retener el 95% de la calidad del modelo completo de 1536 dimensiones. Evalúa siempre con Recall@K en tus datos reales, no con el número de dimensiones como proxy.*

Los embeddings son el fundamento matemático de todo el sistema RAG: sin ellos, la búsqueda semántica no es posible. La siguiente sección presenta los modelos de embedding más relevantes en producción en 2025 —sus características técnicas, sus costos y sus perfiles de rendimiento— para que puedas tomar una decisión informada antes de comenzar la indexación.

Módulo 6 – Capítulo 02 – Sección 02

Modelos de embedding: text-embedding-3, voyage-3, BGE, nomic-embed

El mercado de modelos de embedding en 2024–2025 es amplio y activo, con nuevas versiones y nuevos competidores que aparecen con frecuencia. Para un AI Engineer que debe elegir el modelo de embedding para un sistema de producción, la abundancia de opciones puede ser paralizante. Esta sección presenta los modelos más relevantes con sus características técnicas específicas y los criterios de selección que permiten tomar una decisión fundamentada en lugar de depender del marketing de los proveedores o de rankings abstractos.

Los modelos de OpenAI **text-embedding-3-small** y **text-embedding-3-large** introdujeron en 2024 una característica importante: soporte nativo para Matryoshka Representation Learning (MRL), que permite comprimir el vector de salida a dimensiones menores —de 1536 a 256, o de 3072 a 256— con degradación mínima de calidad. En la práctica, text-embedding-3-small con 256 dimensiones MRL ofrece rendimiento comparable a text-embedding-ada-002 (el modelo anterior de OpenAI) en la mayoría de los dominios, con un costo de almacenamiento 6x menor. La API acepta hasta 2048 textos por llamada en array, lo que permite paralelizar eficientemente la indexación de corpus grandes. text-embedding-3-large, con 3072 dimensiones, es la opción de mayor calidad pero su costo (\$0.13/M tokens vs. \$0.02/M para el small) solo se justifica para dominios donde esos 2–3 puntos adicionales en MTEB se traducen en mejoras de Recall@K medibles.

Voyage AI emerge en 2024 como el competidor más serio a OpenAI en el mercado de embeddings de API. voyage-3 lidera los benchmarks de recuperación en dominios legales, financieros y científicos con un MTEB de 68.32 en la subcategoría de retrieval, superior al text-embedding-3-large en esos dominios. Su característica más diferenciadora es la ventana de contexto de 32K tokens, que permite embedir documentos completos sin fragmentarlos —especialmente valioso para contratos legales o artículos científicos donde el significado depende del documento completo—. A \$0.006/M tokens (menos de un tercio del precio de text-embedding-3-small), voyage-3 es también la opción más económica entre los modelos de API de alta calidad. Voyage AI además ofrece modelos especializados: voyage-law-2 para

documentos legales, voyage-finance-2 para documentos financieros y voyage-code-3 para código fuente, con rendimiento significativamente superior a los modelos generales en esos dominios específicos.

El ecosistema **open source** ofrece alternativas que eliminarán por completo la dependencia de proveedores externos y el riesgo de deprecación de modelos. **BGE-M3** del BAAI (Beijing Academy of Artificial Intelligence) es el modelo open source más versátil: soporta simultáneamente dense retrieval (embeddings vectoriales estándar), sparse retrieval (embeddings dispersos estilo SPLADE) y ColBERT retrieval (embeddings por token en lugar de por documento), todo desde un único modelo de 500M parámetros. Puede hostearse localmente con aproximadamente 4GB de VRAM, tiene soporte para más de 100 idiomas y está disponible bajo licencia Apache 2.0. **nomic-embed-text-v1.5** es la alternativa open source más ligera: 137M parámetros, soporte MRL, contexto de 8192 tokens y disponible via Ollama para inferencia local sin GPU de alta gama. Para equipos que quieren embeddings en su propia infraestructura sin el overhead de un servidor de modelos, nomic-embed es frecuentemente la opción de menor fricción.

Comparación técnica de modelos principales

Modelo	Dim	Tokens Contexto	MTEB Retrieval	Precio/M tokens	Hosting
text-embedding-3-small	1536 (MRL: 256–1536)	8191	~62	\$0.02	API OpenAI
text-embedding-3-large	3072 (MRL: 256–3072)	8191	~65	\$0.13	API OpenAI
voyage-3	1024	32000	68.32	\$0.006	API Voyage
BGE-M3	1024	8192	~64	\$0	Self-hosted
nomic-embed-text-v1.5	768 (MRL)	8192	~62	\$0	Self-hosted / Ollama

- **text-embedding-3-small**: opción estándar para prototipado y sistemas de bajo volumen; integración nativa con LangChain y LlamaIndex; soporte MRL para reducción de dimensiones; límite de 3000 requests/min (ampliable con acuerdo empresarial).
- **voyage-3**: opción de mayor calidad en recuperación para dominios especializados; ventana de 32K tokens; modelos especializados por dominio; recomendado para sistemas

legales, financieros o científicos donde el 5% adicional de Recall@K justifica el cambio.

- **BGE-M3**: opción más versátil en open source; soporte de dense, sparse y ColBERT desde un modelo; multilingue de alta calidad; requiere infraestructura de serving (Hugging Face Text Embeddings Inference o similar).
- **nomie-embed-text-v1.5**: opción de menor fricción para hosting local; disponible via Ollama sin configuración; adecuado para corpus menores de 1M documentos con restricciones de soberanía de datos.

***Nota del Arquitecto:** Un error frecuente es seleccionar el modelo de embedding en producción basándose en el ranking global de MTEB sin validar en el dominio específico. He visto proyectos donde text-embedding-3-large (el modelo con mejor MTEB global) tenía un Recall@5 3 puntos inferior a voyage-3 en el corpus específico del cliente porque ese corpus era de contratos legales en español, dominio donde voyage-3 tiene entrenamiento especializado. Siempre construir un mini-benchmark con 100 pares (query, chunk_relevante) antes de tomar la decisión final de modelo.*

La elección del modelo de embedding es una decisión de largo plazo: cambiar el modelo en producción implica reindexar todo el corpus, un proceso que puede tardar horas o días y requiere gestión cuidadosa de la transición. La Sección 6 de este capítulo cubre el proceso de selección empírica que minimiza el riesgo de esta decisión. Antes, la Sección 3 explora cuándo los modelos generales son insuficientes y el fine-tuning de embeddings para dominios específicos es la respuesta.

Módulo 6 – Capítulo 02 – Sección 03

Embeddings de dominio específico vs embeddings generales

Los modelos de embedding generales como text-embedding-3-large son entrenados sobre corpus web masivos —Common Crawl, Wikipedia, libros digitalizados, código fuente de GitHub— y tienen representaciones sólidas para lenguaje natural cotidiano. Pero en dominios donde el vocabulario, las relaciones conceptuales y los patrones semánticos difieren sustancialmente del lenguaje general, la brecha de calidad puede ser significativa y difícil de compensar con otras optimizaciones del pipeline RAG. Esta brecha —el domain gap— es el punto de partida para entender cuándo invertir en embeddings especializados y cuándo el modelo general es suficiente.

El ejemplo más claro del domain gap es la terminología médica. Términos como "troponina I cardíaca", "QTc prolongado", "creatinina sérica" o "insuficiencia renal estadio 4 por CKD" aparecen con baja frecuencia en el corpus web general comparado con un corpus de PubMed o de historias clínicas. Un modelo general tratará "troponina" como un término poco frecuente, posiblemente fragmentando su tokenización, y producirá un embedding que no captura adecuadamente su relación semántica con "marcadores de daño miocárdico" o "infarto agudo de miocardio". Esto se traduce directamente en un Recall@K más bajo en sistemas RAG médicos: queries sobre cardiopatías no encuentran los documentos relevantes porque el modelo de embedding no representa bien la similitud semántica entre la terminología del médico (la query) y la terminología del artículo médico (el corpus).

La solución más común y efectiva es el **fine-tuning contrastivo** de un modelo base sobre datos del dominio. El proceso usa triplas (query, chunk_positivo, chunk_negativo) del corpus específico, donde el chunk positivo contiene la respuesta a la query y el chunk negativo es semánticamente similar pero no relevante. La librería `sentence-transformers` implementa este proceso en pocas decenas de líneas de código usando `MultipleNegativesRankingLoss`: el modelo aprende a acercar el embedding de la query al del chunk positivo y a alejarlos del chunk negativo. Con 1.000–10.000 triplas de calidad, el fine-tuning puede mejorar el Recall@5 en 10–20 puntos sobre el modelo base en el dominio específico, un delta que raramente se alcanza optimizando otros componentes del pipeline.

La técnica de **hard negative mining** es crítica para que el fine-tuning produzca modelos de alta calidad. Los negativos "fáciles" —chunks sobre temas completamente distintos al chunk positivo— ya son bien manejados por el modelo base sin fine-tuning. Lo que el fine-tuning necesita aprender son las distinciones semánticas finas del dominio: "insuficiencia renal" vs. "insuficiencia hepática" son conceptos médicos que un modelo general puede no distinguir bien en el espacio vectorial porque comparten mucho vocabulario común. El hard negative mining selecciona como negativos los chunks que el modelo base actual recupera incorrectamente —los que tienen mayor similitud coseno con la query pero no son relevantes— forzando al modelo a aprender exactamente las distinciones que el corpus específico requiere.

Para equipos sin recursos para hacer fine-tuning propio, Voyage AI ofrece la alternativa práctica: modelos pre-especializados por dominio —voyage-law-2 para documentos legales en inglés, voyage-finance-2 para análisis financiero, voyage-code-3 para código fuente— que capturan las relaciones semánticas específicas de esos dominios sin requerir datos de entrenamiento adicionales. El delta de calidad de voyage-law-2 sobre text-embedding-3-large en corpus de contratos legales es de aproximadamente 5–8 puntos en Recall@10, una mejora significativa que puede lograrse simplemente cambiando el proveedor de embeddings.

Aspectos técnicos del dominio específico

- **Domain gap:** distancia entre la distribución de texto del corpus de entrenamiento del modelo y la distribución del corpus de producción; se cuantifica comparando Recall@K del modelo en MTEB vs. en un benchmark interno con queries y documentos del dominio objetivo.
- **Fine-tuning contrastivo con sentence-transformers:** proceso de ajuste de pesos del modelo con `MultipleNegativesRankingLoss` usando pares (query, documento_relevante); requiere mínimo 1.000 pares de calidad; coste computacional de horas en una GPU T4 para modelos base de 110M parámetros.
- **Hard negative mining:** seleccionar como negativos de entrenamiento los chunks que el modelo actual recupera pero son incorrectos; implementable con ANN retrieval + filtrado de qrels; es la diferencia entre un fine-tuning mediocre y uno que supera al modelo base en 15+ puntos.
- **Vocabulary coverage:** verificar qué porcentaje del vocabulario técnico del corpus está tokenizado eficientemente; términos técnicos fragmentados en muchos subword tokens producen embeddings de menor calidad; modelos con tokenizadores entrenados en el dominio tienen cobertura superior.

- **Benchmark interno:** dataset de evaluación con 100–500 pares (query, documentos_relevantes) anotados por expertos del dominio; herramienta de medición del delta entre modelo general y especializado; prerequisite para justificar el esfuerzo de fine-tuning.
- **Modelos de código:** voyage-code-3 y los modelos CodeBERT tratan el código como un dominio semántico distinto; identificadores de variables, nombres de funciones y comentarios determinan la similitud semántica; los modelos generales fallan en recuperación de snippets equivalentes escritos con nomenclatura diferente.

La decisión entre modelo general, modelo especializado de API y fine-tuning propio se reduce a tres preguntas: ¿cuál es el delta de calidad de Recall@K en mi benchmark interno? ¿Cuál es el costo del fine-tuning vs. el costo del modelo especializado de API a escala de mi volumen de corpus? ¿Tengo los datos de entrenamiento necesarios para un fine-tuning de calidad? Con las respuestas a estas preguntas sobre datos reales, la decisión se toma sola. La siguiente sección profundiza en la dimensionalidad de los vectores y la mecánica de la similitud coseno, conceptos que afectan directamente las decisiones de almacenamiento y rendimiento del índice.

Módulo 6 – Capítulo 02 – Sección 04

Dimensionalidad, normalización y similitud coseno

Cuando un modelo de embedding produce un vector de 1536 dimensiones para un chunk de texto, ¿qué significa cada número en ese vector? La respuesta corta es que no existe una interpretación directa por dimensión: el espacio latente no organiza las dimensiones como "eje de formalidad", "eje de positividad" o cualquier otra categoría humana reconocible. Las dimensiones son combinaciones distribuidas de patrones estadísticos aprendidos del corpus de entrenamiento, y su significado es colectivo, no individual. Lo que sí tiene significado directo y cuantificable es la geometría del espacio: la distancia angular entre dos vectores es la métrica de similitud semántica en la que se basa toda la recuperación de información en un sistema RAG.

La **dimensionalidad** de un vector de embedding tiene implicaciones concretas en tres dimensiones operativas: almacenamiento, latencia de búsqueda y calidad de recuperación. Un vector float32 de 1536 dimensiones ocupa 6KB en memoria; un índice de 10 millones de chunks de ese tamaño requiere 60GB de VRAM, lo que impone restricciones de infraestructura significativas. La tecnología de **Matryoshka Representation Learning (MRL)**, desarrollada por el equipo de OpenAI para text-embedding-3 y adoptada también por nomic-embed-v1.5, resuelve este problema de forma elegante: entrena el modelo de forma que los primeros N componentes del vector ya contengan la información semántica más importante, permitiendo truncar el vector a 256 o 512 dimensiones con una caída de calidad en MTEB de apenas 2–3 puntos. Para un corpus de 10 millones de chunks, la reducción de 1536 a 256 dimensiones disminuye el requisito de memoria de 60GB a 10GB con pérdida de calidad mínima.

La **normalización L2** es una operación que transforma el vector a longitud unitaria dividiendo cada componente por la norma euclidiana del vector: $v_{norm} = v / ||v||_2$. Esta operación tiene una consecuencia matemática importante: la similitud coseno entre dos vectores normalizados es idéntica a su producto punto. Esto significa que las bases vectoriales optimizadas para producto punto (que pueden usar operaciones BLAS y SIMD altamente optimizadas en CPU y cuBLAS en GPU) producen exactamente el mismo ranking que una

búsqueda por similitud coseno, pero con latencia significativamente menor. La mayoría de las bases vectoriales modernas (Qdrant, Pinecone, pgvector) asumen que los vectores están normalizados y producen resultados incorrectos si se insertan sin normalizar; es uno de los errores de configuración más comunes en implementaciones iniciales de RAG.

La **similitud coseno** mide el coseno del ángulo entre dos vectores en el espacio de alta dimensionalidad, con valores en el rango $[-1, 1]$: 1 significa vectores idénticos en dirección (máxima similitud semántica), 0 significa vectores ortogonales (sin relación semántica), y -1 significa vectores opuestos (raramente observado en embeddings de texto porque los vectores suelen residir en el cuadrante positivo del espacio latente). Su complemento, la distancia coseno = $1 - \text{similitud coseno}$, convierte la similitud en una métrica de distancia con rango $[0, 2]$ usable en índices de búsqueda. La similitud coseno es la métrica correcta para comparar embeddings de texto porque es **invariante a la magnitud del vector**: un chunk de 50 tokens y uno de 500 tokens sobre el mismo tema pueden tener magnitudes muy distintas (el vector del chunk más largo tiende a tener mayor norma), pero la similitud coseno captura su proximidad semántica independientemente de esa diferencia de magnitud.

Aspectos técnicos de dimensionalidad y métricas

- **Matryoshka Representation Learning (MRL)**: técnica de entrenamiento que permite truncar el vector de 3072 a 256 dimensiones manteniendo la mayor parte de la información semántica; disponible en text-embedding-3 de OpenAI y nomic-embed-v1.5; la reducción de 8x en dimensionalidad reduce el almacenamiento y la latencia de búsqueda ANN proporcionalmente.
- **Normalización L2**: operación $\mathbf{v} / \|\mathbf{v}\|_2$ que convierte el vector a longitud unitaria; habilita optimizaciones SIMD y BLAS que aceleran la búsqueda; requisito previo a la inserción en la mayoría de las bases vectoriales; verificar siempre que el modelo de embedding devuelve vectores ya normalizados o normalizar explícitamente.
- **Producto punto vs. similitud coseno**: equivalentes para vectores L2-normalizados; el producto punto es preferible en implementación por su eficiencia computacional; sin normalización, el producto punto favorece vectores de mayor magnitud independientemente del ángulo, produciendo rankings incorrectos.
- **Distancia euclidiana (L2)**: alternativa usada en ciertos modelos de imagen o audio; generalmente inferior a similitud coseno para texto porque la magnitud del embedding depende de la longitud del texto, introduciendo un sesgo sistemático hacia chunks más largos.

- **Curse of dimensionality en ANN:** a medida que la dimensionalidad crece, la diferencia entre la distancia al vecino más cercano y al más lejano tiende a cero, degradando la discriminabilidad del índice; modelos con dimensiones superiores a 2048 pueden mostrar Recall@K inferior a modelos de 768 dimensiones en índices HNSW sobre los mismos datos.
- **Análisis PCA de embeddings:** proyección de los vectores del índice a 2D o 3D para diagnosticar visualmente la calidad del índice; vectores colapsados (baja varianza entre todos los vectores), clusters espurios o distribuciones anómalas son señales de problemas en el pipeline de ingesta o en la normalización.

Nota del Arquitecto: Un error frecuente en implementaciones iniciales es no verificar si el modelo de embedding elegido devuelve vectores ya normalizados o sin normalizar. OpenAI text-embedding-3 devuelve vectores normalizados por defecto; sentence-transformers con mean pooling no normaliza por defecto a menos que se especifique `normalize_embeddings=True`. Insertar vectores sin normalizar en Qdrant configurado para producto punto produce rankings completamente incorrectos sin ningún mensaje de error explícito. La verificación tarda un segundo: `assert abs(np.linalg.norm(embedding) - 1.0) < 1e-6`.

La comprensión de la dimensionalidad y la métrica de similitud no es solo teoría: determina decisiones prácticas de arquitectura como cuánta memoria de GPU necesita el índice, si MRL puede reducir los costos de almacenamiento aceptablemente, y si el modelo de embedding elegido requiere normalización explícita antes de la inserción. Con estos fundamentos establecidos, la siguiente sección cierra el capítulo con el proceso de evaluación y selección del modelo de embedding correcto para el dominio específico del sistema.

Módulo 6 – Capítulo 02 – Sección 05

Evaluación de modelos de embedding: MTEB benchmark y métricas de recuperación

Elegir el mejor modelo de embedding para un sistema RAG sin datos de evaluación es lo mismo que elegir el mejor medicamento sin ensayos clínicos: la intuición y los rankings globales son proxies imperfectos de lo que realmente importa, que es el rendimiento en el dominio y el tipo de queries específicos del sistema. El Massive Text Embedding Benchmark (MTEB) es el estándar de referencia de la industria, pero usarlo incorrectamente lleva a decisiones equivocadas. Esta sección explica qué mide MTEB, cómo interpretarlo correctamente y, más importante, cómo construir y ejecutar la evaluación propia sobre los datos reales del sistema.

El **MTEB**, publicado originalmente por Muennighoff et al. (2022) y mantenido como leaderboard público en [hf.co/spaces/mteb/leaderboard](https://huggingface.co/spaces/mteb/leaderboard), evalúa modelos de embedding en 56 datasets distribuidos en 8 categorías de tareas: retrieval, classification, clustering, pair classification, reranking, summarization, bitext mining y STS (Semantic Textual Similarity). El ranking global agrega el rendimiento en todas las categorías con igual peso, pero para RAG la categoría relevante es **Retrieval**, que comprende 15 datasets del benchmark BEIR (Benchmarking Information Retrieval): BioASQ para biomedicina, HotpotQA para razonamiento multi-hop, FiQA para finanzas, NFCorpus para ciencias médicas, ArguAna para argumentación, SciFact para verificación de claims científicos, entre otros. Un modelo que lidera el ranking global de MTEB pero tiene rendimiento mediocre en el subcategory Retrieval es un candidato cuestionable para un sistema RAG.

La interpretación correcta del MTEB requiere identificar qué subdatasets del benchmark Retrieval son más similares al dominio del sistema. Si el corpus del sistema es documentación técnica de software, ArguAna y TREC-COVID son menos predictivos que un benchmark de código o Stack Overflow. Si el corpus es legal, los datasets BEIR más predictivos son FiQA (razonamiento financiero, con estructura similar a textos legales) y SciFact (verificación de afirmaciones factuales específicas). El benchmark MTEB es un buen punto de partida para

reducir la lista de candidatos de 20 a 4-5 modelos, pero la decisión final siempre debe basarse en evaluación interna sobre datos reales.

Las **métricas de recuperación** que se usan tanto en MTEB como en la evaluación interna tienen semántica específica que es importante comprender antes de interpretarlas. **Recall@K** es la métrica primaria para RAG: mide qué fracción de los documentos relevantes para cada query están presentes entre los K documentos recuperados. En el caso más común (una única respuesta correcta por query), Recall@K equivale a "¿el chunk con la respuesta está entre los K primeros?"; un Recall@5 de 0.80 significa que para el 80% de las queries el sistema recupera el chunk correcto entre los cinco primeros. **MRR (Mean Reciprocal Rank)** pondera inversamente la posición del primer hit correcto: un sistema que siempre pone el chunk correcto en primera posición tiene MRR=1; si el correcto suele aparecer en tercera posición, el MRR es aproximadamente 0.33. MRR es especialmente relevante cuando el LLM da más peso a los primeros chunks del contexto. **NDCG@K** es la métrica más completa: maneja relevance gradada (distintos niveles de relevancia para distintos documentos), pondera logarítmicamente la posición y normaliza por el ranking perfecto teórico.

Métricas de recuperación y su interpretación

- **Recall@K (K=1, 3, 5, 10, 20)**: métrica primaria para RAG; K=5 es el valor estándar para sistemas que envían 5 chunks al LLM; Recall@5 > 0.75 es el umbral mínimo para producción; Recall@20 mide el ceiling de calidad antes del reranking.
- **Precision@K**: fracción de los K recuperados que son relevantes; trade-off inverso con Recall@K; relevante cuando el número de tokens de contexto es limitado y cada slot de chunk tiene alto costo; Precision@5 > 0.60 indica que la mayoría de los chunks enviados al LLM son pertinentes.
- **MRR (Mean Reciprocal Rank)**: rango entre 0 y 1; MRR=1 si el primer resultado siempre es el correcto; útil para sistemas de QA de respuesta única; complementa Recall@1 al proporcionar información sobre la distribución de posiciones del hit correcto.
- **NDCG@K (Normalized Discounted Cumulative Gain)**: soporta relevance gradada (0=irrelevante, 1=parcialmente relevante, 2=muy relevante); estándar en BEIR y en evaluaciones de búsqueda empresarial; fórmula $DCG = \sum(\text{rel}_i / \log_2(i+1))$ normalizado por el DCG ideal.
- **Hit Rate**: variante simplificada de Recall; si al menos un documento relevante está entre los K recuperados, es un hit; idéntico a Recall@K cuando hay un único documento relevante por query; más fácil de comunicar a stakeholders no técnicos.

- **Construcción del dataset de evaluación:** pares (query, [chunk_ids_relevantes]) anotados manualmente por expertos del dominio o generados con LLM + validación; mínimo 100 queries para resultados estadísticamente significativos; la representatividad del dataset determina la validez de las métricas obtenidas.

Nota del Arquitecto: En tres proyectos de RAG empresarial con dominios distintos (legal, técnico y científico), el modelo que lideraba el leaderboard global de MTEB fue el seleccionado inicialmente en todos los casos. En todos los casos, la evaluación interna sobre el corpus real reveló que un modelo diferente —voyage-3 en el legal, BGE-M3 en el técnico, voyage-3 en el científico— superaba al líder de MTEB en 5–12 puntos de Recall@5. El corolario: siempre construir el dataset de evaluación interno en la primera semana del proyecto, antes de tomar ninguna decisión de modelo.

La evaluación de modelos de embedding no es un ejercicio único al inicio del proyecto: es un proceso que debe repetirse cada vez que el corpus o el perfil de queries cambia significativamente, o cuando el proveedor del modelo lanza una nueva versión. Los modelos de embedding son una dependencia versionada del sistema RAG, y cambiarlos implica reindexar todo el corpus; esa fricción hace que la evaluación inicial rigurosa sea una inversión que se amortiza durante toda la vida del sistema. La siguiente sección cierra el capítulo con los criterios para seleccionar el modelo de embedding correcto, incluyendo consideraciones de lock-in, actualización y riesgo de proveedor.

Módulo 6 – Capítulo 02 – Sección 06

Cierre: elegir el modelo de embedding correcto para el dominio del problema

La elección del modelo de embedding es la decisión técnica con mayor impacto en la calidad de todo el sistema RAG porque establece el techo de lo que el retriever puede lograr: ninguna técnica de búsqueda híbrida, reranking o expansión de queries puede recuperar lo que el embedding ha perdido al proyectar el texto al espacio vectorial. Si el modelo de embedding no captura que "insuficiencia renal crónica estadio 3" y "CKD G3" son sinónimos exactos en el dominio médico, la búsqueda semántica fallará sistemáticamente en ese tipo de queries, y el reranking no puede corregir lo que el retriever no ha recuperado. Esta asimetría entre la importancia de la decisión y el esfuerzo requerido para tomarla bien —un proceso de evaluación sistemático de días, no semanas— es la razón por la que muchos proyectos RAG se lanzan con el modelo de embedding incorrecto: la evaluación interna parece opcional hasta que las métricas de calidad en producción muestran que no lo era.

El proceso de selección correcto sigue cuatro pasos que deben ejecutarse en orden. Primero, construir el dataset de evaluación interno: 100–500 pares (query, [chunk_ids_relevantes]) representativos del uso real del sistema, anotados por expertos del dominio si es posible o generados con LLM y validados manualmente para los casos más importantes. Este dataset es el activo más valioso del proceso de selección y se reutilizará en cada iteración del sistema. Segundo, identificar los candidatos razonables a partir del leaderboard de MTEB Retrieval: típicamente 4–6 modelos que incluyen el líder general (actualmente voyage-3 en muchos dominios), el mejor open source (BGE-M3 o nomic-embed-v1.5), el modelo de OpenAI (text-embedding-3-large), y si existe, el modelo especializado en el dominio (voyage-law-2, voyage-finance-2, voyage-code-3). Tercero, ejecutar la evaluación de todos los candidatos sobre el dataset interno con las métricas definidas (Recall@5, NDCG@10, latencia p99) y ordenarlos. Cuarto, seleccionar el modelo que muestra el mejor balance entre rendimiento de recuperación, costo por millón de tokens, latencia y riesgo de lock-in al proveedor.

La consideración del **lock-in** es frecuentemente subestimada en la selección inicial. Los modelos de API como OpenAI text-embedding-3 o Voyage AI voyage-3 pueden ser

deprecados por el proveedor, forzando una reindexación completa del corpus; OpenAI deprecated text-embedding-ada-002 en 2024, y los equipos que habían indexado millones de documentos con ese modelo tuvieron que planificar una migración costosa. Los modelos open source como BGE-M3 y nomic-embed-v1.5, alojados en infraestructura propia bajo licencias Apache 2.0, eliminan este riesgo: el modelo siempre está disponible bajo control del equipo, independientemente de las decisiones del proveedor. El precio de este control es la necesidad de infraestructura de serving (una GPU de al menos 8GB VRAM para BGE-M3) y la responsabilidad de gestionar las actualizaciones del modelo.

En producción, el modelo de embedding debe tratarse como un **componente versionado del sistema**: el índice vectorial está acoplado a la versión específica del modelo que generó sus vectores, y cambiar el modelo implica reindexar todo el corpus con el nuevo modelo antes de poder usar el nuevo índice en producción. Esta dependencia de versión debe registrarse explícitamente en los metadatos de cada vector (campo `embedding_model_version` en el payload de Qdrant o Pinecone) para poder auditar qué modelo generó qué parte del índice y coordinar las migraciones correctamente.

Criterios de selección del modelo de embedding

Criterio	Opciones a considerar
Rendimiento en dominio (Recall@5 en dataset interno)	Evalúate todos los candidatos; diferencias de 5–10 puntos son decisivas
Costo por millón de tokens en ingesta	text-embedding-3-small (\$0.02) vs. voyage-3 (\$0.06) vs. BGE-M3 (self-hosted ~\$0.001)
Costo por millón de tokens en serving	Mismo cálculo que ingesta, multiplicado por el volumen diario de queries
Riesgo de lock-in al proveedor	Open source (cero lock-in) vs. API propietaria (riesgo de deprecación)
Latencia de embedding de queries en serving	API: 50–200ms; modelo local GPU: 5–20ms; modelo local CPU: 50–500ms
Dimensionalidad y uso de memoria del índice	256 (MRL truncado) a 3072; impacto directo en el costo de la base vectorial

Nota del Arquitecto: El criterio de selección más ignorado es la latencia de embedding de queries en tiempo de serving, no en ingesta. Un modelo que tarda 200ms en embedir cada query del usuario añade esa latencia a cada consulta del sistema RAG, impactando directamente la experiencia del usuario. Para sistemas con SLO de latencia total <1 segundo, un modelo de embedding que consume 200ms del presupuesto es un problema desde el día uno.

La selección del modelo de embedding correcta, respaldada por evaluación empírica en el dominio específico, es la inversión técnica de mayor retorno en las primeras semanas de un proyecto RAG. Los capítulos siguientes construyen sobre esta base: el Capítulo 3 analiza las bases de datos vectoriales que almacenan y sirven los embeddings; el Capítulo 4 aborda el chunking que determina qué texto se embide; y el Capítulo 5 desarrolla las estrategias de recuperación que usan los embeddings para encontrar el conocimiento correcto.

"En cualquier sistema de recuperación de información, la calidad de la representación es el límite superior de la calidad de la recuperación; ninguna mejora algorítmica posterior puede superar la información perdida en la proyección al espacio latente." — Gerard Salton, pionero de la recuperación de información moderna y creador del modelo vectorial espacial.